

Yumia

Un linguaggio dichiarativo per documenti visivi

[Nome e Cognome]" align="center

Il problema delle presentazioni tecniche

Perché creare slide tecniche è complesso e inefficiente

Approccio Tradizionale (WYSIWYG / Manuale)

TOOL VISUALI & SLIDE EDITOR

- Disposizione manuale di box di testo e forme
- Formattazione incoerente che degrada nel tempo
- File binari opachi non versionabili con Git
- Rischio costante di disallineamenti tra formati

vs

Problemi & Limiti Strutturali

ATTRITI DI SVILUPPO & AI

- Zero riutilizzabilità tra documentazione e slide
- Frammentazione tra Markdown, HTML e PPTX
- Gli LLM faticano a generare layout geometrici stabili
- Nessun test automatico di qualità visiva

La soluzione: Il Design Compiler

Dall'intento comunicativo alla composizione automatica

Intento vs Geometria

Non diciamo al computer 'disegna un rettangolo a coordinate (x,y)', ma 'crea una Card per evidenziare una funzionalità'.

Componenti Semantici

Primitive di alto livello: hero, metric, card, grid, compare, timeline, chart e diagram.

Layout Deterministico

Un motore di calcolo automatico distribuisce spazi, gap, contrasti e gerarchie visive senza overflow.

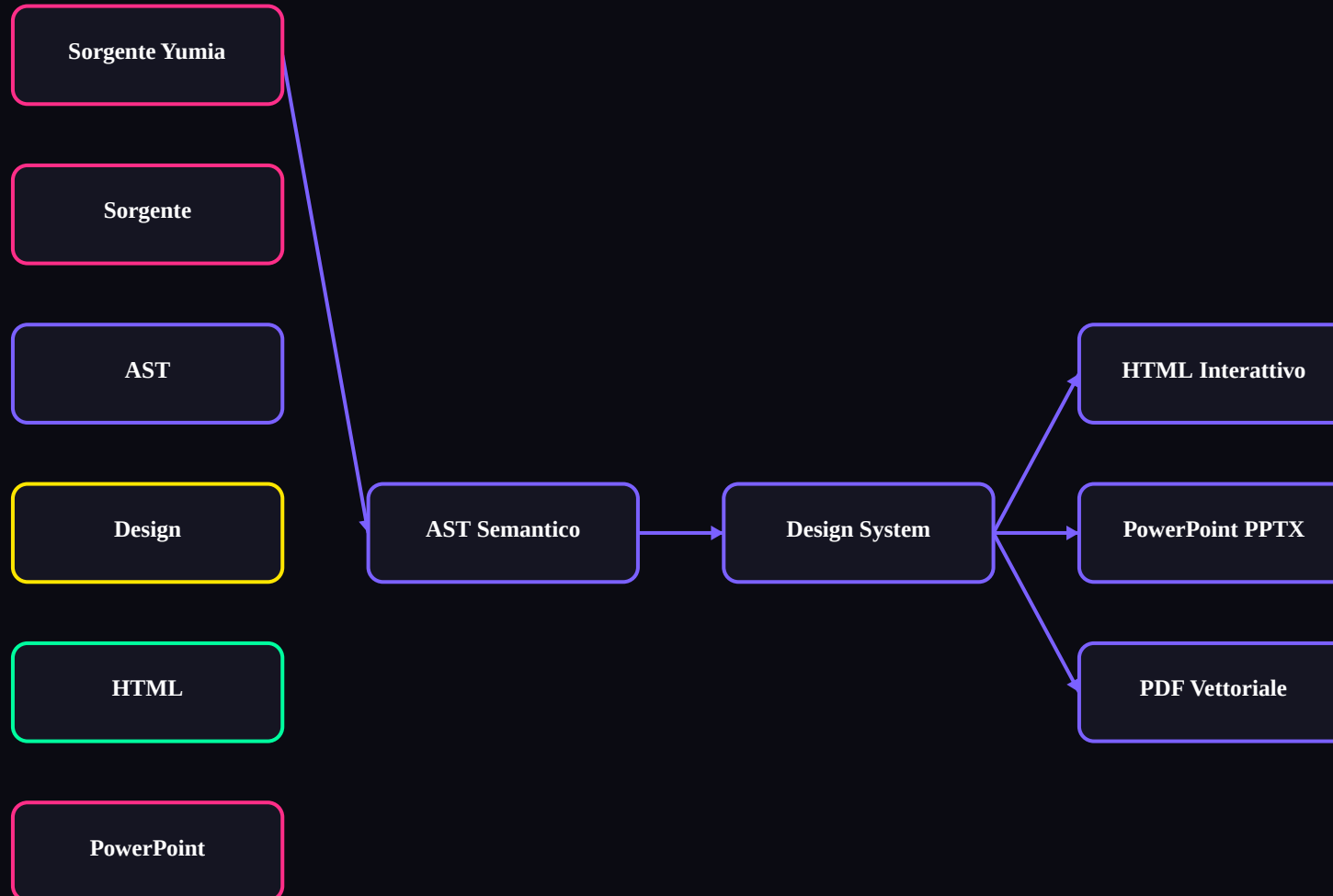
Cambio di paradigma

Yumia separa radicalmente il contenuto dal rendering visivo: lo sviluppatore scrive la semantica, il compilatore applica il design system.

Un'unica sorgente, più formati

Compilazione Multi-Target da una Singola Sorgente

Flusso di Trasformazione Multi-Target

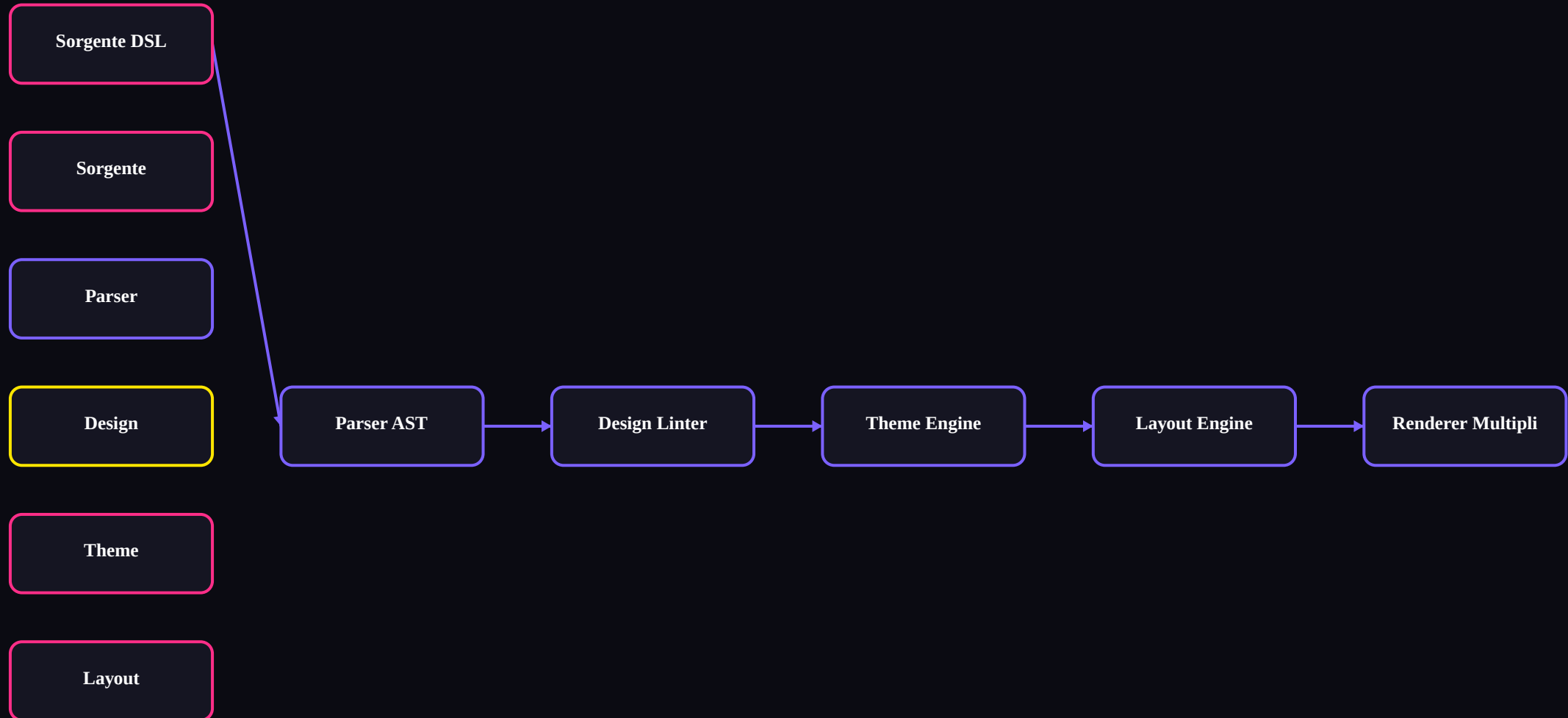


Principio Single Source of Truth

Architettura della Pipeline

Dalla sintassi al rendering finale: la pipeline a 7 stadi

Pipeline di Compilazione Deterministica



Pipeline deterministica a passate isolate

L'AST e il Design Intent

Rappresentazione intermedia del significato e della struttura

AST: Abstract Syntax Tree

Struttura dati intermedia ad albero che incapsula il contenuto, la gerarchia logica e i vincoli semantici di presentazione.

EMPHASIS

DENSITY

HIERARCHY

ALIGN

Componenti Semantici Astratti

- Struttura: Hero, Heading, Paragraph, Columns, Grid
- Dati & Logica: Metric, Card, Compare, Timeline
- Visuali: Diagram, Sequence, Class, Chart, Table
- Annotazioni: Badge, Callout, Math, Notes

Esempio di Codice Native Yumia

Sintassi pulita, dichiarativa e basata su indentazione

```
1 document "Yumia Demo"
2 theme "cyberpunk"
3 aspectRatio "16:9"
4 slide "Yumia in azione"
5 hero title="Design Compiler" subtitle="Dall'intento al documento
6   visible" badge="Demo"
7   grid columns="3" gap="20"
7 metric "1" label="Sorgente"
8 metric "3" label="Formati"
9 metric "0" label="Layout manuale"
```

Caratteristiche del linguaggio

- Struttura gerarchica pulita basata sull'indentazione
- Componenti semantici senza CSS a basso livello
- Nessun tag HTML di chiusura o sintassi verbosa
- Perfettamente leggibile e modificabile in qualsiasi editor

I Tre Output Principali

Generazione nativa e specializzata per ogni ecosistema

HtmlRenderer

- Presentazione web interattiva SPA
- Navigazione da tastiera e fullscreen
- Modalità Speaker View con note
- Inspector visuale in tempo reale

PptxRenderer

- Forme OpenXML native e testo modificabile
- Tabelle, card e grafici nativi Office
- Zero screenshot o immagini statiche
- Modificabile su PowerPoint / Keynote

PdfRenderer

- Rendering vettoriale ad alta definizione
- Layout e tipografia rigorosamente fissati
- Ideale per dispense, esami e stampa
- Dimensioni del file ottimizzate

Design Linter e Qualità Visiva

Static Analysis per la leggibilità e l'accessibilità

Controlli Automatici di Design

- Densità informativa e carico cognitivo
- Verifica slide vuote o incomplete
- Contrasto testo/sfondo secondo standard WCAG
- Rispetto della Safe Area e margini
- Calcolo del Visual Score da 0 a 100

Esempio di Diagnostica CLI

```
1 [YUM004] Alta densità informativa
2 La slide contiene troppi elementi complessi.
3 Suggerimento: dividere la slide o usare
4 una griglia di metriche.
```

Prevenzione prima della build

Il linter intercetta difetti grafici, errori tipografici e problemi di leggibilità prima di distribuire la presentazione.

CLI e Workflow di Sviluppo

L'esperienza di sviluppo pensata per sviluppatori e CI/CD

1. Scrittura & Validazione

```
1 yumia validate pres.yumia
2 yumia explain pres.yumia
```

2. Live Dev & Audit

```
1 yumia dev pres.yumia --open
2 yumia check --optimize
```

3. Build & Deploy

```
1 yumia build --format pptx
2 yumia deploy --provider gh-pages
```

Workflow integrabile in CI/CD

Ogni commit su Git può validare le slide, generare la documentazione HTML e pubblicare la versione aggiornata su GitHub Pages.

Yumia e Intelligenza Artificiale

Perché i linguaggi dichiarativi semantici superano l'HTML grezzo

AI con HTML / CSS Grezzo

FRAGILE & IMPREVEDIBILE

- Allucinazioni di coordinate pixel e layout rotti
- Codice verboso con rischio di sintassi non valida
- Difficile da vincolare a un design system coerente
- Revisione manuale complessa per l'utente

vs

AI con Yumia DSL

ROBUSTO & STRUTTURATO

- Generazione guidata da JSON Schema (yumia schema)
- Sintassi concisa focalizzata sul contenuto semantico
- Il motore garantisce allineamenti e layout perfetti
- Validazione deterministica automatica post-generazione

Funzionalità e Componenti Avanzati

Un ricco ecosistema integrato senza dipendenze esterne

Componenti & Macro

Creazione di blocchi riutilizzabili parametrici con la direttiva component.

Data Binding

Iterazione dinamica su dataset JSON o CSV per reportistica automatica.

Formule Matematiche

Rendering di formule scientifiche complesse in sintassi LaTeX / KaTeX.

Compatibilità e Percorso di Migrazione

Continuità garantita tra ecosistema Markdown e Native DSL

Markdown Yumia (.yumia.md)

```
1 :::metric value="99%" label="Uptime" change="+1%" :::
```

Compatibile con lettori Markdown standard, esteso tramite direttive semantiche.

Native Yumia (.yumia)

```
1 metric "99%" label="Uptime" diff="+1%"
```

Sintassi nativa più concisa, pulita e ottimizzata per i Design Compiler.

Utility di Migrazione Automatica

Il compilatore include tool di migrazione bidirezionale per consentire un passaggio progressivo senza perdere i contenuti esistenti.

Vantaggi, Limiti e Sviluppi Futuri

Valutazione dello stato attuale e roadmap del progetto

Vantaggi Attuali

- Sorgente 100% versionabile con Git
- Tre output compilati da un unico file
- Layout e gerarchia visiva garantiti
- Linter automatico per design e WCAG
- Integrazione nativa con workflow AI

Aspetti da Consolidare

- Parità totale di rendering su edge case
- Estensione della copertura dei test
- Supporto a font e asset esterni complessi
- Documentazione di scenari avanzati

Sviluppi Futuri

- Editor visuale WYSIWYG bidirezionale
- Collaborazione in tempo reale su browser
- Integrazione con modelli LLM in locale
- Plugin per VS Code ed estensioni IDE

Descrivere ciò che si vuole comunicare,

lasciare al compilatore il compito di comporlo.

1. Separazione

Il contenuto è completamente disaccoppiato dalle coordinate grafiche.

2. Flessibilità

Un singolo sorgente produce Web interattivo, PowerPoint nativo e PDF.

3. Futuro

Un ponte naturale tra sviluppatori, designer e intelligenza artificiale.

Sessione Q&A

Grazie per l'attenzione. Spazio per domande, chiarimenti e feedback della commissione.